

Project 1 Report

Tarika Gupta

1 Data Creation Methodology

We created a 20-example golden Q&A set for the RapidFire AI documentation by systematically varying questions along three dimensions: *feature*, *persona*, and *scenario*. The feature dimension captured the task being asked about, such as RAG configuration, vector store setup, experiment management, dashboard usage, troubleshooting, or tutorial-level understanding. The persona dimension captured the type of user, including first-time users, API integrators, experimenters, tuners, and infrastructure-oriented users working over SSH. The scenario dimension captured the form of the request, such as factual lookup, conceptual explanation, procedural setup, or comparison/decision-making.

This design made the dataset more systematic than an ad hoc collection of questions. For each candidate tuple, we identified the relevant documentation section, wrote a realistic user query, drafted a grounded reference answer, and then manually checked the item for clarity, non-triviality, duplication, and overlap with the released validation set. Each finalized item was paired with exact file-and-line evidence spans.

The final dataset contains 20 questions and 25 total source-evidence spans, for an average of 1.25 spans per question. It covers 8 unique source files, with most evidence concentrated in instruction-dense files such as `ragspecs.rst`, `walkthroughrag.rst`, `experiment.rst`, and `dashboard.rst`. The average question length is 16.55 words, and the average reference-answer length is 55.25 words. The dataset is also balanced across the three design dimensions: it includes questions on RAG configuration, installation and remote workflows, experiment execution, dashboard usage, troubleshooting, tutorials, and online aggregation.

We intentionally emphasized API-heavy and failure-prone parts of the documentation, since these are the cases where retrieval quality, evidence attribution, and grounded generation matter most. We avoided overly trivial single-field lookups and instead favored questions requiring precise grounding or synthesis within a documentation section.

As one representative example, we used the tuple (*remote setup for RAG experimentation, first-time user on a remote machine, procedural request*) to create the question: “If you installed RapidFire AI on a remote machine over SSH, which ports do you need to forward to access the RAG workflow?” The reference answer was grounded in `walkthroughrag.rst`, lines 107–120, and required extracting the exact ports 8850, 8851, and 8853. This item is useful because it tests whether the system can retrieve a compact procedural span and generate a precise, non-hallucinated answer.

Overall, our methodology was to start from a structured tuple, convert it into a realistic documentation question, and manually validate that the final answer was precise, source-grounded, and useful for evaluating both retrieval and generation failures.

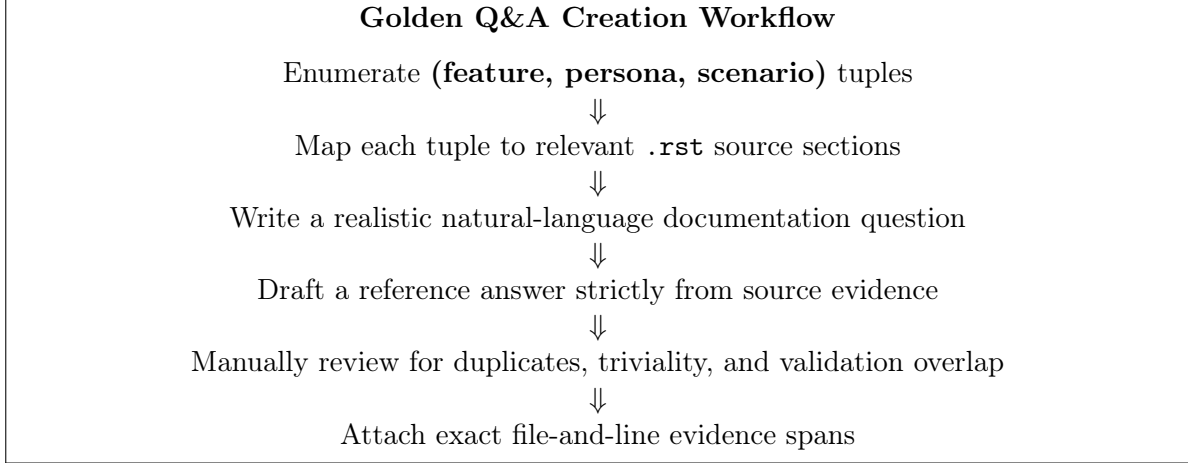


Figure 1: Structured workflow used to create the golden Q&A dataset.

2 Final System Architecture

Our final system is a single-turn RAG pipeline over the RapidFire AI documentation corpus, implemented in `main.py`. Given a JSON file of questions, the system outputs a JSON file containing the generated answer, the retrieved context passed to the generator, and structured file-and-line source spans. The main design goal was to improve retrieval and generation quality while strictly respecting the 2,000-token per-query context budget.

The pipeline first loads the released `.rst` documentation files and splits them into overlapping line-based chunks of roughly 160 words. Each chunk stores its file name, line span, text, tokenized form, and a lightweight heading signal. Retaining exact line ranges allows retrieved chunks to be converted directly into the required source-span format.

For retrieval, we use a hybrid strategy. The lexical component is a BM25-style scorer over normalized word tokens, with small boosts for heading and filename matches. This helps capture exact terminology, parameter names, and file-specific language. In parallel, we embed both chunks and queries using TritonAI’s `api-tgpt-embeddings` model, which helps retrieve relevant documentation even when the question is phrased differently from the source text.

We merge the lexical and embedding candidate pools using reciprocal-rank-style fusion, preserving both exact-match and semantically similar candidates. The merged candidates are then reranked with `api-mistral-small-3.2-2506`, which is prompted to prioritize chunks that directly answer the question and contain concrete details such as parameter names, defaults, constraints, and procedural steps. This improves the quality of the final evidence passed to the generator.

Before generation, the system filters redundant chunks and serializes the remaining selected chunks with explicit source markers of the form `[SOURCE: filename lines start-end]`. We enforce the 2,000-token limit by estimating the full prompt cost, including the system prompt, user prompt, separators, and retrieved context, then packing chunks only until the remaining budget is exhausted.

Finally, the generator uses TritonAI’s OpenAI-compatible API with `api-mistral-small-3.2-2506`. The prompt instructs the model to answer only from the retrieved context, include concrete details when available, and avoid unsupported claims. The final output contains `question_id`, `answer`, `retrieved_context`, and `sources`, supporting both the released retrieval evaluator and the LLM judge’s faithfulness checks.

Overall, the final architecture balances lexical precision, semantic recall, reranking quality, redundancy-aware context packing, and strict prompt-budget enforcement under a TritonAI-only

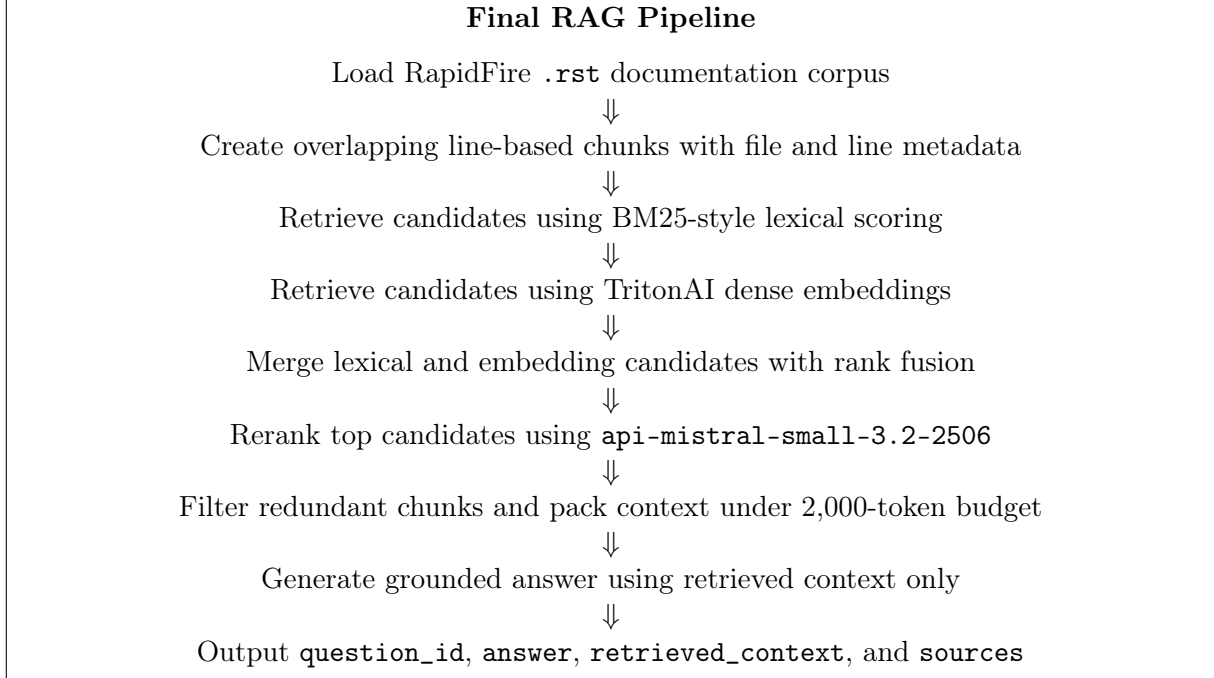


Figure 2: Overview of the final single-turn RAG pipeline.

setup.

3 Experimentation Methodology

We used RapidFire to compare retrieval configurations for the Project 1 RAG pipeline while keeping `main.py` as the final single-configuration submission script. Since retrieval determines which evidence is available to the generator, the sweep focused primarily on retrieval quality, followed by a generation check on the best-performing variant.

The baseline configuration used 160-word chunks, 2-line overlap, 10 lexical candidates, 10 embedding candidates, 8 rerank candidates, TritonAI embeddings, an LLM reranker, and a 2,000-token prompt budget with a 64-token safety margin. RapidFire services were started on the server, and the sweep was run as follows:

```
conda activate cse234
rapidfireai start
python run_rapidfire.py
```

The sweep tested 8 configurations against `validation-set-golden-qa-pairs.json`. Each run produced a Project-1-style output JSON under `logs/rapidfire_runs/`, evaluated retrieved source spans with the released retrieval metrics, and logged the run to RapidFire/MLflow. The retrieval score was computed as the mean of F1@5, Precision@5, and Recall@5.

The best retrieval-only configuration was `final_top3`, which kept the same chunk size, candidate pools, embeddings, and reranker as the default system but reduced the final selected context from 4 chunks to 3. This increased the retrieval score from 0.6928 to 0.7408, mainly by improving precision while keeping recall high. Smaller chunks also improved precision, and broader candidate pools improved recall, but neither outperformed `final_top3`. The ablations confirmed that both

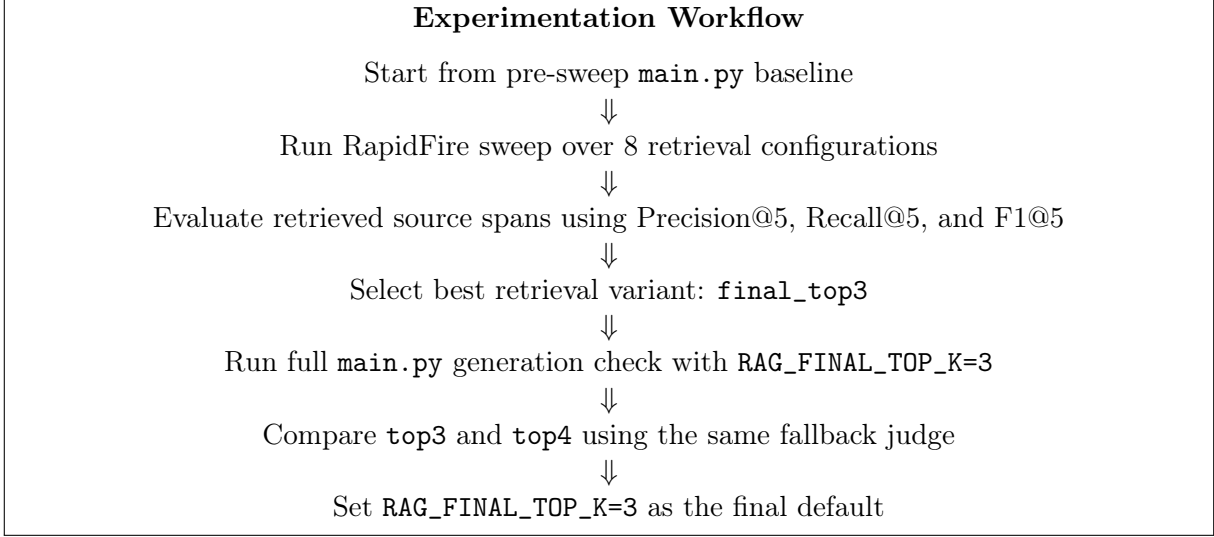


Figure 3: RapidFire-based tuning workflow for selecting the final RAG configuration.

Configuration	Description
<code>final_top3</code>	Same as the default hybrid system, but returns the top 3 final chunks instead of 4.
<code>smaller_chunks_120_top4</code>	Uses smaller 120-word chunks while keeping the final top-k at 4.
<code>broader_candidates_top4</code>	Expands the lexical and embedding candidate pools before reranking, increasing candidate recall.
<code>final_default_160_top4</code>	Pre-sweep default system: 160-word chunks, hybrid retrieval, LLM reranking, and top 4 final chunks.
<code>larger_chunks_220_top4</code>	Uses larger 220-word chunks while keeping the final top-k at 4.
<code>final_top5</code>	Same as the default hybrid system, but returns the top 5 final chunks instead of 4.
<code>hybrid_no_reranker_top4</code>	Uses both lexical and embedding retrieval, but disables the LLM reranker.
<code>lexical_only_top4</code>	Uses only BM25-style lexical retrieval, with embeddings and reranking disabled.

Table 1: Description of the retrieval configurations tested in the RapidFire sweep.

embeddings and reranking were useful: `lexical_only_top4` and `hybrid_no_reranker_top4` were clearly worse than the full hybrid reranked system.

After the sweep, we ran the full pipeline with the selected setting:

```
RAG_FINAL_TOP_K=3 python main.py \
--input validation-set-golden-qa-pairs.json \
--output logs/final_top3_validation_output.json \
--corpus-dir sourcedocs \
--apikey-txt ~/api-key.txt \
--generation-model api-mistral-small-3.2-2506
```

The released retrieval evaluator gave a retrieval score of 0.7447, with F1@5 = 0.6915, Precision@5 = 0.5815, and Recall@5 = 0.9611. The official `claude-sonnet-4-6-aws` judge could not produce a reliable generation score because the API hit a daily token limit, causing 28 judge failures out of 45 questions. Therefore, for a controlled comparison, we used the documented fallback judge, `api-gpt-oss-120b`, on both `top3` and the previous `top4` default.

Configuration	Retrieval Score	F1@5	Precision@5	Recall@5
final_top3	0.7408	0.6871	0.5741	0.9611
smaller_chunks_120_top4	0.7279	0.6801	0.5815	0.9222
broader_candidates_top4	0.7157	0.6435	0.5148	0.9889
final_default_160_top4	0.6928	0.6211	0.4852	0.9722
larger_chunks_220_top4	0.6576	0.5820	0.4519	0.9389
final_top5	0.6442	0.5552	0.4163	0.9611
hybrid_no_reranker_top4	0.6189	0.5383	0.4019	0.9167
lexical_only_top4	0.5997	0.5158	0.3778	0.9056

Table 2: Retrieval results from the RapidFire configuration sweep.

Configuration	Retrieval Score	Partial Generation Score	Average
final_top3	0.7447	0.8919	0.8183
final_default_160_top4	0.6928	0.8637	0.7783

Table 3: Same-judge comparison between the selected **top3** configuration and the previous **top4** default.

Under this same-judge comparison, **final_top3** improved both retrieval and partial generation score, so we changed `main.py` to use `RAG_FINAL_TOP_K=3` as the default. The value remains configurable through the `RAG_FINAL_TOP_K` environment variable.

Overall, the experiments showed that the best score came from balancing retrieval precision, semantic recall, reranking quality, and context discipline. Adding more chunks did not necessarily help; in this validation set, fewer but more relevant chunks produced cleaner evidence and better judged answers. The final configuration therefore uses hybrid lexical-plus-embedding retrieval, LLM-based reranking, redundancy-aware context packing, and strict full-prompt budget enforcement.

4 Results and Analysis

We evaluated all configurations on the released validation set using both the retrieval metrics and the LLM-judge metrics. Since the leaderboard weights retrieval and generation equally, we compared systems using retrieval score, partial generation score, and their average. This helped us avoid optimizing retrieval overlap at the expense of answer quality.

The lexical baseline showed that a simple BM25-style retriever could already produce strong judged answers when the retrieved context was clean and focused. However, more aggressive section-aware retrieval did not improve the overall score: although it slightly improved retrieval overlap, its generation score dropped, suggesting that the selected context became noisier or less useful for answer synthesis.

The largest improvement came from adding hybrid retrieval and reranking. By combining lexical BM25-style candidates with Triton embedding candidates, the system handled both exact technical lookups and more conceptually phrased questions. The LLM reranker further improved evidence quality by promoting chunks that directly contained parameter values, defaults, procedural steps, or decision-relevant details. This raised the average score from the lexical baseline range of roughly 0.76 to 0.8002.

The best result came from the final budget-aware tuned pipeline, which achieved a retrieval score of 0.6843, a partial generation score of 0.9644, and an average score of 0.8244. Its retrieval breakdown was $\text{Precision@5} = 0.4796$, $\text{Recall@5} = 0.9611$, and $\text{F1@5} = 0.6122$. Its generation breakdown was

Configuration	Retrieval Score	Partial Generation Score	Average
Earlier lexical baseline run	0.5856	0.9348	0.7602
Restored old baseline	0.5932	0.9200	0.7566
Section-aware / refined retriever	0.6052	0.8830	0.7441
Hybrid retrieval + reranking	0.6552	0.9452	0.8002
Final budget-aware tuned pipeline	0.6843	0.9644	0.8244

Table 4: Main validation-set results used to select the final system.

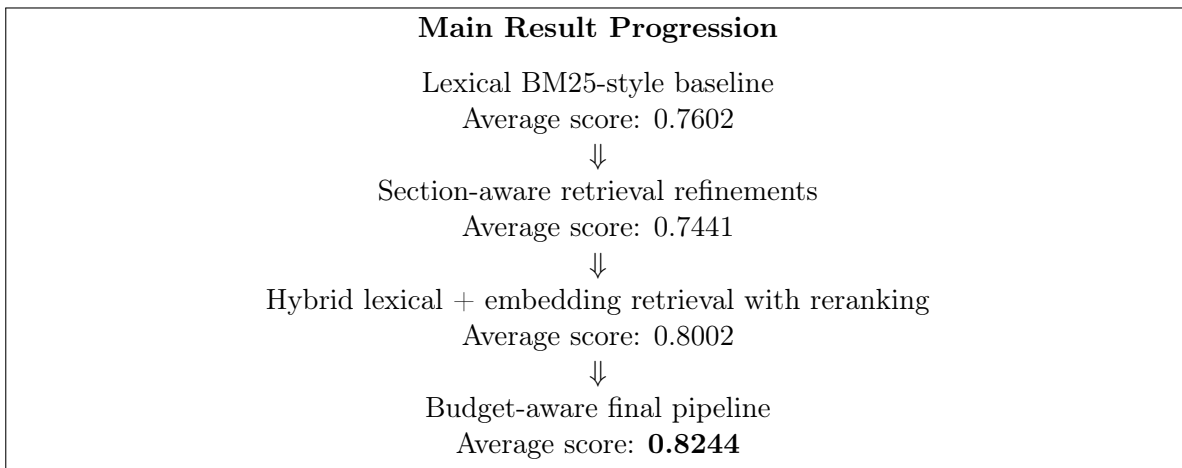


Figure 4: Progression from the initial baseline to the final tuned system.

Correctness = 1.0000, Faithfulness = 1.0000, and Completeness = 4.4667 / 5. These results show that the final system retrieved relevant evidence for most questions while also producing grounded and complete answers.

Overall, the experiments showed that RAG quality depends on the interaction between retrieval, reranking, context packing, and prompt design. Increasing recall alone was not sufficient; the final context also had to be precise, non-redundant, and small enough to fit cleanly within the 2,000-token prompt budget. The final system performed best because it balanced semantic recall, lexical precision, reranking quality, and strict budget enforcement.

The strongest configuration was not the one that maximized retrieval recall alone, but the one that produced the cleanest answer-bearing context under the token budget. Hybrid retrieval, LLM reranking, redundancy filtering, and full-prompt budget enforcement jointly produced the best overall score.

5 AI Tools Statement

We used AI coding assistants as development aids during this project, primarily to speed up implementation, debugging, documentation cleanup, and iterative refinement of our RAG pipeline. In particular, AI assistance was used to help restructure retrieval code, test alternative prompt and context-packing strategies, improve README and submission compliance, and draft report language from our actual experimental results and design decisions.

However, the core project decisions remained human-directed. We chose the evaluation strategy, interpreted the released metrics, selected which configurations to test, compared tradeoffs between

retrieval and generation quality, and decided which final pipeline to submit. We also manually verified that the final system satisfied the project constraints, including output schema requirements, token-budget enforcement, TritonAI compatibility, and autograder-facing CLI behavior.

We treated AI tools as accelerators for implementation and writing rather than as replacements for engineering judgment. All final code, experiments, scores, and report claims were reviewed and validated by us before inclusion in the submission.